# Indian Sign Language Recognition & Speech

A hands-only, isolated-sign recognition prototype with a local browser interface.

**Prepared for mentor discussion | 26 October 2026**

## Project objective

Recognize selected Indian Sign Language (ISL) gestures from a webcam, display their corresponding words and speak confirmed results aloud. The current system is a small-vocabulary prototype intended to establish the collection, training and application pipeline.

## Current scope at a glance

| Item | Verified status |
|---|---|
| Vocabulary | Five signs: HELLO, THANK_YOU, YES, NO and BYE; plus a silent NO_SIGN class. |
| Initial dataset | 180 valid landmark clips: 30 per class, each containing 30 frames. |
| Model evaluation | 22 of 24 saved test clips correct (91.7%); small internal random clip split. |
| Interface | Local Streamlit app with webcam, text, history and offline Windows speech. |
| Main issue | NO can be mistaken for NO_SIGN, resulting in silence. Targeted extra collection is prepared; completion and retraining are not confirmed. |

## Suggested opening explanation

"We have built the end-to-end prototype from landmark collection to live text and speech output. Our current focus is separating confusing classes and validating real-world performance before expanding the vocabulary."

# Background and approach

## Why sign recognition is a sequence problem

A sign can depend on hand shape, orientation, location and movement over time. Classifying a single image can miss movement information. This project therefore gives the model a sequence of landmark frames rather than a single webcam image.

## Computer vision front end

OpenCV reads the webcam and mirrors the image horizontally, as in the collection script. MediaPipe Hand Landmarker locates up to two hands. Each detected hand provides 21 landmarks, with x, y and z values for each landmark. These are landmark coordinates, not calibrated physical distances.

## Input representation

**2 hands x 21 landmarks x 3 values = 126 features per frame.** The first 63 values represent the hand labelled Left by MediaPipe and the next 63 represent Right. An absent hand contributes zeros. A clip contains 30 frames, giving an input shape of **(30, 126)**. Collection and recognition share the same feature extraction function.

## Why an LSTM is used

A Long Short-Term Memory network is a recurrent neural network that processes ordered sequences. Here it learns how hand landmark values change across the recorded movement. It is a baseline for this small vocabulary; we have not established that it is the best architecture for a larger ISL system.

## Why NO_SIGN is necessary

A classifier trained only on signs still selects a known class when given ordinary hand movements. NO_SIGN supplies examples of visible resting hands and non-sign activity. It must remain distinct from the meaningful sign NO. Low-confidence predictions are also withheld, although confidence alone does not guarantee correctness.

## Scope boundaries

The implementation currently uses hand landmarks only. It does not extract pose or face landmarks, interpret grammatical facial expressions, translate continuous signing into sentences, or generate sentence grammar. The word history is a list of confirmed outputs, not sentence translation. The planned 30-sign vocabulary is not yet implemented.

# Technology stack and libraries

Versions below were read from the local training/app environment and requirement files. They document this project setup, not the latest public releases.

| Technology / version | Role in this project |
| --- | --- |
| Python 3.12 training/app environment | Runs model training, webcam recognition and the Streamlit app. Virtual environments isolate dependencies. An earlier camera environment used Python 3.14. |
| TensorFlow 2.21.0 / tf.keras | Builds and trains the LSTM, embeds normalization, saves .keras models, and runs predictions. tf.function compiles inference to reduce repeated execution overhead. |
| MediaPipe 1.1.0 | Hand Landmarker in VIDEO mode; detects up to two hands using a local hand_landmarker.task asset. |
| opencv-contrib-python 5.0.0.93 (cv2) | Webcam access, image mirroring, BGR-to-RGB conversion, landmark drawing and native collection windows. |
| NumPy 2.5.4 | Stores numeric features, builds sequence arrays, saves/loads .npy clips and computes evaluation statistics. |
| Streamlit 1.66.1 | Provides the local browser UI, settings, controls, preview and recognized-word display. |
| pywin32 312 + Windows SAPI | Accesses the Windows speech engine through COM for offline spoken output. pythoncom manages COM initialization in the speech thread. |
| Python standard library | threading, multiprocessing and concurrent.futures separate capture, startup and inference; deque holds rolling windows; pathlib, json, argparse, time and uuid support storage and scripts. |
| Git and GitHub | Track source changes and release tags. The working initial prototype was pushed and tagged v1.0.0. |

## Hardware and operating model

The prototype runs locally on a Windows laptop with a webcam. Current native-Windows TensorFlow execution uses the CPU in this setup. The local app is reached at 127.0.0.1:8501. It is not a hosted service or a packaged mobile app. Speech and saved-model inference run locally after dependencies and assets are available.

# Dataset and training pipeline

## Collection workflow

The recorder shows a three-second countdown, captures 30 landmark frames and saves a clip as a UUID-named .npy file under data/<LABEL>/. It waits 50 ms after each processed recording frame, so the complete clip duration also includes processing and camera overhead. Thirty frames should not be described as an exact fixed number of seconds.

Validation requires the expected shape, finite values and hand visibility in at least 80% of frames. Invalid old clips are rejected during loading. Existing valid clips count toward a target, so collection can resume without overwriting recordings.

## Initial data split

The released model used 30 clips for each of six classes. The seeded random split assigns 22 training, four validation and four test clips per class: **132 training / 24 validation / 24 test**. Whole clips are split; frames from one clip are not split across sets. There is no signer-independent or recording-session-independent evaluation.

## Model architecture

| Layer / step | Configuration and purpose |
|---|---|
| Input | 30 frames x 126 landmark features |
| Normalization | Feature statistics learned from training data only; normalization saved inside the model |
| LSTM | 64 units to summarize the sequence |
| Dropout | 0.3 to reduce overfitting during training |
| Dense hidden layer | 32 units with ReLU activation |
| Output layer | Six units with softmax, one probability per class |

## Training configuration

Adam optimizer; sparse categorical cross-entropy loss; batch size 16; default maximum 80 epochs; seed 42. Early stopping monitors validation loss with patience 12 and restores the best weights. Each training run saves its model and a JSON report in a new timestamped directory.

## Targeted collection now prepared

The recorder supports --signs NO NO_SIGN --target 60. With 30 existing clips per selected class, this requests 30 more valid clips for each. New collection completion has not been verified. Retraining must be followed by per-class evaluation and live tests before selecting a replacement model.

# Application architecture and source files

| Pipeline stage | What happens |
|---|---|
| 1. Webcam capture | A dedicated thread reads and mirrors camera frames so preview capture can continue while recognition is busy. |
| 2. Landmark tracking | A local worker process runs MediaPipe. Shared feature extraction produces the ordered 126-value vector. |
| 3. Temporal window | The recognition loop maintains a rolling 30-frame buffer. Brief tracking gaps are tolerated; sustained loss clears the window. |
| 4. Model prediction | The same worker process runs the saved LSTM through compiled inference. The parent receives class probabilities. |
| 5. Confirmation | A prediction must pass the 0.8 threshold and agree across three consecutive prediction windows; inference is attempted every five valid frames once the buffer is full. |
| 6. Text and speech | Confirmed sign words appear in the UI and history. The speech gate suppresses repeats. NO_SIGN is silent; speech can be muted. |

## Why separate model processing from the UI?

Model loading and inference previously caused startup or preview stalls. Camera capture now runs separately, and the TensorFlow/MediaPipe worker is isolated from Streamlit rerun cleanup. Windows multiprocessing startup is protected by the app entry-point guard. A cached worker avoids reloading the model on every camera restart.

## Main source files

| File | Responsibility |
|---|---|
| collect_data.py / dataset.py | Recording, resumable targets, vocabulary, validation and dataset loading |
| hand_features.py | Shared left/right landmark ordering |
| train_model.py | Training, split creation, evaluation and model/report saving |
| recognize.py / speech.py | Original CLI recognizer, prediction confirmation and offline speech |
| app.py / app_runtime.py | Streamlit layout, controls, camera ownership and recognition loop |
| inference_worker.py | Persistent model/tracker process and local pipe communication |
| start_app.bat / requirements-*.txt | Windows launcher and environment dependencies |

# Results, limitations and next milestones

| Class | Correct / test clips | Recall |
|---|---|---|
| HELLO | 4 / 4 | 100% |
| THANK_YOU | 3 / 4 | 75% |
| YES | 4 / 4 | 100% |
| NO | 3 / 4 | 75% |
| BYE | 4 / 4 | 100% |
| NO_SIGN | 4 / 4 | 100% |

**Overall: 22/24 correct (91.7%).** One THANK_YOU clip was predicted as HELLO; one NO clip was predicted as NO_SIGN. These are saved internal test results, not a measured live-user accuracy score.

## Performance and verification

A local benchmark measured median eager model inference at about 127 ms and compiled inference at about 2.4 ms, with matching saved-test class predictions. Those timings exclude the time to perform and collect a sign. Earlier live checks observed about 30 camera frames/s and 13-16 tracking frames/s; subsequent input-timing corrections mean those are historical observations, not a fresh end-to-end latency guarantee.

Nine automated checks passed during app debugging, covering controls, worker execution, cleanup, silent background behavior and responsiveness during slow loading/prediction. They do not establish recognition accuracy on real gestures.

## Current limitation and practical next steps

**Immediate:** complete targeted NO/NO_SIGN collection; train a separate candidate model; inspect class-level errors; test NO audio and resting-hand silence. Check all existing signs for regressions. Preserve the v1 model while testing.

**Next:** collect balanced, varied examples for additional signs, document recording sessions and reserve independent sessions/users for testing. Measure live success rate, false positives, missed signs and time to confirmed output. Expand toward 30 signs only after the smaller vocabulary is reliable.

**Later:** assess pose/face cues, continuous-sign segmentation, sentence-level processing and deployment requirements as separate extensions. These are future tasks, not completed capabilities.

## Project record and evidence

Repository: github.com/sahanabhat14/sign_launguage_thing. Initial release: v1.0.0. App work is unreleased in the project record; no new GitHub release is claimed here.

Technical evidence: requirements-app.txt, requirements-live.txt, requirements-training.txt, dataset.py, train_model.py, recognize.py, app_runtime.py, inference_worker.py, speech.py and models/v1.0.0/report.json. Status reflects verified source and recorded project work as of 26 October 2026.